Phase 4: Security Reduction and Known Attacks

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 14, 2026

Outline: Phase 4

1. The Approximate-GCD Problem
2. The Security Reduction (Outline)
3. Why Standard Attacks Fail
4. The Geometric Threat: Lattice Attacks
5. References

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must rigorously prove that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must rigorously prove that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

For RSA, that problem is integer factorization. For Gentry's 2009 FHE, it is finding short vectors in ideal lattices.

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must rigorously prove that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

For RSA, that problem is integer factorization. For Gentry's 2009 FHE, it is finding short vectors in ideal lattices.

For our integer-based scheme, the foundation is the **Approximate-GCD Problem**.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^{ρ}.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^ρ.

Definition (Approximate Integer Greatest Common Divisor)

The (ρ, η, γ)-approximate-gcd problem is: given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^ρ.

Definition (Approximate Integer Greatest Common Divisor)

The (ρ, η, γ)-approximate-gcd problem is: given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

Intuition

If the noise $r = 0$, the problem is trivial. You are given exact multiples of p, and a single run of the 2,000-year-old Euclidean algorithm computes $\gcd(x_1, x_2) = p$ instantly. The addition of the noise term r is the sole cryptographic barrier.

Theorem 4.2: The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem 4.2: The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem (4.2)

Any attack $\mathcal{A}$ with advantage ϵ on the semantic security of the encryption scheme can be converted into an algorithm $\mathcal{B}$ for solving the (ρ, η, γ)-approximate-gcd problem with success probability at least $\epsilon/2$.

Theorem 4.2: The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem (4.2)

Any attack $\mathcal{A}$ with advantage ϵ on the semantic security of the encryption scheme can be converted into an algorithm $\mathcal{B}$ for solving the (ρ, η, γ)-approximate-gcd problem with success probability at least $\epsilon/2$.

The Setup: Algorithm $\mathcal{B}$ is given random noisy multiples of a hidden integer p. Its goal is to find p. To do this, $\mathcal{B}$ uses the attacker $\mathcal{A}$ as a tool, feeding $\mathcal{A}$ simulated public keys and ciphertexts to extract information about p.

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

Step 1: The Learn-LSB Oracle

- $\mathcal{B}$ constructs randomized, simulated ciphertexts from a noisy integer z.
- By observing whether $\mathcal{A}$ correctly guesses the encrypted bits, $\mathcal{B}$ can deduce the **parity of the quotient** $q_p(z)$, without ever knowing p!

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

Step 1: The Learn-LSB Oracle

- $\mathcal{B}$ constructs randomized, simulated ciphertexts from a noisy integer z.
- By observing whether $\mathcal{A}$ correctly guesses the encrypted bits, $\mathcal{B}$ can deduce the **parity of the quotient** $q_p(z)$, without ever knowing p!

Step 2: The Binary GCD Extraction

- $\mathcal{B}$ takes two noisy multiples of p and runs a standard Binary-GCD algorithm on them.
- Whenever the algorithm needs to know if a value is even or odd, $\mathcal{B}$ queries the Learn-LSB Oracle.
- **Result:** Iterating this process strips away the noise terms completely, reducing the values down to the exact secret key p.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled (Section 4.1.1):

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled (Section 4.1.1):

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.
2. **Noise Drowning:** $\mathcal{B}$ injects a massive random noise term (drawn from $2^{\rho'}$) into the simulation. This completely "drowns out" and statistically hides any underlying discrepancies in the base noise.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled (Section 4.1.1):

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.
2. **Noise Drowning:** $\mathcal{B}$ injects a massive random noise term (drawn from $2^{\rho'}$) into the simulation. This completely "drowns out" and statistically hides any underlying discrepancies in the base noise.

Intuition

Because the simulated data is statistically indistinguishable from real FHE traffic, $\mathcal{A}$'s advantage holds. A rigorous counting argument proves that $\mathcal{B}$ will successfully recover the Approximate-GCD secret key p with probability $\geq \epsilon/2$.

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

The Cross-Noise Barrier

The distance is dominated by the cross-multiplied noise term $(q_2 r_1 - q_1 r_2)/p$. Because this distance is significantly larger than $1/q_2^2$, Dirichlet's approximation theorem does not apply. The target fraction is hopelessly obscured by the noise.

The Simultaneous Diophantine Approximation (SDA)

Because simple algebraic attacks fail, attackers must turn to the geometry of numbers—specifically lattice reduction algorithms like LLL and BKZ.

The Simultaneous Diophantine Approximation (SDA)

Because simple algebraic attacks fail, attackers must turn to the geometry of numbers—specifically lattice reduction algorithms like LLL and BKZ.

Given samples $x_0, x_1, \ldots, x_t$, we construct a $(t+1)$-dimensional SDA lattice L spanned by the rows of matrix M:

$$M = \begin{pmatrix} 2^\rho & 0 & 0 & \ldots & 0 \\ x_1 & -x_0 & 0 & \ldots & 0 \\ x_2 & 0 & -x_0 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_t & 0 & 0 & \ldots & -x_0 \end{pmatrix}$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

The Critical Cancellation: Look at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$q_0 x_i - q_i x_0 = q_0(q_i p + r_i) - q_i(q_0 p + r_0)$$

$$= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0$$

$$= \mathbf{q_0 r_i - q_i r_0}$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

The Critical Cancellation: Look at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$q_0 x_i - q_i x_0 = q_0(q_i p + r_i) - q_i(q_0 p + r_0)$$

$$= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0$$

$$= \mathbf{q_0 r_i - q_i r_0}$$

The Threat

The secret key p perfectly cancels out! This leaves $\vec{v}$ as a relatively short target vector in the lattice. If an algorithm like LLL can find $\vec{v}$, the attacker recovers the quotients, computes $p \approx x_0/q_0$, and the scheme is broken.

Defeating the Lattice

Will LLL or BKZ actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Defeating the Lattice

Will LLL or BKZ actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Scenario 1: Attacker uses a small number of samples t

- By Minkowski's bound, a low-dimensional lattice contains exponentially many *spurious* vectors that are much shorter than our target $\vec{v}$.
- The target vector is completely buried in geometric noise.

Defeating the Lattice

Will LLL or BKZ actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Scenario 1: Attacker uses a small number of samples t

- By Minkowski's bound, a low-dimensional lattice contains exponentially many *spurious* vectors that are much shorter than our target $\vec{v}$.
- The target vector is completely buried in geometric noise.

Scenario 2: Attacker uses a massive number of samples t

- The attacker forces t to be huge so $\vec{v}$ is the absolute shortest vector.
- However, known lattice reduction algorithms take time $\approx 2^{t/k}$ to find a 2^k approximation.
- Isolating $\vec{v}$ strictly requires time roughly $\mathbf{2}^{\gamma/\eta^2}$.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise r.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth for bootstrapping).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Specifically ensures that $\gamma = \omega(\eta^2 \log \lambda)$. This makes the time $2^{\gamma/\eta^2}$ super-polynomial, thwarting all SDA and lattice reduction algorithms.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise r.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth for bootstrapping).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Specifically ensures that $\gamma = \omega(\eta^2 \log \lambda)$. This makes the time $2^{\gamma/\eta^2}$ super-polynomial, thwarting all SDA and lattice reduction algorithms.

Conclusion: While this results in massive public keys, it mathematically guarantees rigorous security based solely on the hardness of the Approximate-GCD over the integers.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- J. C. Lagarias. *The computational complexity of simultaneous diophantine approximation problems*. SIAM J. Comput., 1985.
- N. Howgrave-Graham. *Approximate integer common divisors*. CaLC, 2001.